Evaluating ML for EEG-Based Attention State Classification Under Budget Hardware Constraints

CMPT 353 - Project Report

Monir Fathalla

August 6, 2026

1 Introduction and Motivation

I recently joined NeuraXtension, a neuroscience club at Simon Fraser University. The club is currently exploring projects involving brain-computer interfaces (BCIs), and one idea that came up was building an attention state classifier using electroencephalogram (EEG) data. This inspired the topic of my project: I wanted to investigate how well existing classical machine learning methods perform when constrained to the hardware specifications of the NeuroPawn, a budget consumer EEG headset with 8 channels (F3, Fz, F4, C3, C4, Pz, O1, O2) sampling at 125 Hz.

Most published EEG classification research uses medical-grade equipment with 32–64 electrodes at high sampling rates. The question I set out to answer is whether classical ML techniques can still distinguish between focused and unfocused mental states when limited to just 8 channels and a low sampling rate.

2 Data Acquisition

Finding suitable data proved to be one of the most difficult parts of the project. I initially searched Kaggle but found only heavily pre-processed, single-purpose datasets, and for ones that matched attention state the only one I found ended up being poorly structured and subpar in quality. I needed a competent dataset to build a realistic BCI pipeline from scratch.

I eventually found the XJTU-EEG MEMA (Multi-label EEG for Mental Attention states) dataset: 20 subjects, 12 trials each, totalling over 1,060 minutes of raw EEG recordings. However, the dataset was hosted exclusively on Baidu Pan, a Chinese cloud platform that is effectively inaccessible from North America without a local phone number. I found someone on a subreddit willing to download the data from Baidu and transfer it to me. I paid them $16 in Ethereum. The full dataset was approximately 90 GB.

3 Data Cleaning and Preprocessing

The raw MEMA data consists of tab-separated text files inside a `For_graph` directory, organized by subject. Each file contains 32 columns (one per electrode) with hundreds of thousands of rows recorded at 500 Hz.

My preprocessing pipeline (`data_processing.py`) performs four steps:

1. **Channel selection**: I discard 24 of the 32 columns and keep only the 8 channels matching the NeuroPawn's electrode positions.

2. **Downsampling**: I use `scipy.signal.decimate` to reduce the sampling rate from 500 Hz to 125 Hz, applying an anti-aliasing filter automatically.

3. **Bandpass filtering**: A 4th-order Butterworth filter removes frequencies below 0.5 Hz (motion artifacts) and above 45 Hz (electrical noise).

4. **Windowing**: The continuous signal is segmented into 1-second windows with a 50% overlap. This effectively doubles the training dataset size and allows the models to

capture transient brainwave events that might otherwise be split across window boundaries.

Labels are derived from the filenames: `_a` files correspond to "focused" (attentive) states, while `_n` and `_r` files correspond to "unfocused" (neutral/relaxed) states. Rounds 1–3 are designated as training data and round 4 as test data. The processed windows are saved as compressed `.npz` files.

4 Feature Extraction

From each 1-second window, I extract 12 features per channel using `feature_extraction.py`, yielding 96 features total (12 × 8 channels). These include:

- **Band powers** (5 features): Average power spectral density in the Delta (0.5–4 Hz), Theta (4–8 Hz), Alpha (8–13 Hz), Beta (13–30 Hz), and Gamma (30–45 Hz) bands, computed via Welch's method. To reduce noise, Welch's method was configured with a segment length of half the sampling rate (64 samples) to enable segment averaging. Finally, these power values were logarithmically transformed (`log1p`) to compress their heavily right-skewed distributions.
- **Band ratios** (3 features): Alpha/Beta, Theta/Beta, and Theta/Alpha ratios, which prior literature has linked to attention and ADHD markers.
- **Statistical features** (4 features): Mean, standard deviation, skewness, and kurtosis of the raw time-domain signal within each window.

I then apply five feature selection strategies to produce different feature subsets for comparison:

1. **None**: All 96 features retained as a baseline.
2. **ANOVA**: Features where the F-test p-value ≤ 0.05 are kept (average ≈33 features retained).
3. **Feature Importance (FI)**: A Random Forest is trained and features with above-average importance scores are kept (average ≈24 features).
4. **Linear Correlation (LCC)**: Features with above-average absolute Pearson correlation with the label are kept (average ≈27 features). The system measures correlation between each feature and the label, then keeps labels that are above the average correlation (sum of absolute values of all correlations over n)
5. **PCA**: Principal Component Analysis retaining 95% of variance (average ≈39 components). Setting n__components=0.95 instructs the algorithm to automatically keep the minimum number of components necessary to preserve 95% of the original data's information.

5 Experimental Design

I evaluate four classifiers from `scikit-learn` and `LightGBM`: KNN (k=5), Random Forest (100 trees), Decision Tree, and LightGBM (100 estimators). Support Vector Machines (SVM) were initially tested but ultimately removed from the pipeline; the $O(N^3)$ computational complexity caused it to fail to converge on the large cross-subject training sets within a reasonable timeframe. All models are wrapped in a `StandardScaler` pipeline.

Each model is tested under two evaluation scenarios:

- **Subject-Dependent**: Train on a subject's rounds 1–3, test on their round 4. This simulates a calibrated, personalized device.
- **Cross-Subject**: Leave-one-subject-out: train on all data from 19 subjects, test on the held-out subject. This simulates out-of-the-box performance on a new user.

This produces a total of $20 \times 4 \times 5 \times 2 = 800$ individual experiments. To make this computationally feasible, I parallelized the outer subject loop across all CPU cores using `joblib.Parallel`. By utilizing LightGBM's native multi-threading, training wall-clock time was kept to under 1 hour despite the dataset effectively doubling in size due to the overlapping windows.

6 Results

The mean baseline accuracy (always predicting the majority class) is 55.1% for Subject-Dependent and 52.5% for Cross-Subject, confirming the dataset is approximately balanced.

Figure 1: Mean accuracy across 20 subjects, grouped by model, feature method, and scenario.

6.1 Cross-Subject Results

The following table shows the top cross-subject results. Random Forest with LCC-selected features achieved the highest mean accuracy at 58.0%, followed closely by Gradient Boosting with PCA features (57.9%) and Gradient Boosting with LCC features (57.9%). SVM was dropped in later iterations due to scoring below the baseline in previous tests.

Table 1: Top 5 Cross-Subject configurations by mean accuracy.

Model	Features	Accuracy	F1 Score	Avg Train Time (s)
RF	LCC	58.0%	0.556	47.5
GBoosting	PCA	57.9%	0.554	39.4
GBoosting	LCC	57.9%	0.552	103.7
RF	None (96)	57.8%	0.549	321.9
RF	ANOVA	57.8%	0.552	639.3

6.2 Subject-Dependent Results

Performance improved substantially in the personalized scenario. The top-performing pair (Gradient Boosting with all 96 features) reached 66.9%, and Random Forest with all features was close behind at 66.7%. These higher numbers reflect the benefit of overlapping data windows, logarithmic PSD scaling, and occipital channel coverage for subject-calibrated training.

6.3 Key Observations

1. **Tree ensembles dominate.** LightGBM and Random Forest consistently outperformed all other models in both scenarios. These models can internally discover nonlinear interactions between features, which is critical for noisy EEG data.

2. **Raw vs. Selected features.** Keeping all 96 features (`none`) produced the highest accuracy (66.9%) for the subject-dependent scenario. However, for the cross-subject scenario, LCC and PCA offered slight improvements over the raw features, suggesting feature selection helps filter out subject-specific noise when generalizing.

3. **SVM scalability issues.** Due to its $O(N^3)$ complexity and sensitivity to un-transformed outliers, SVM was dropped as it failed to scale to the large, noisy cross-subject dataset within reasonable iteration limits.

4. **Subject-dependent vs. cross-subject gap.** There is roughly a 7–8 percentage point drop from subject-dependent to cross-subject accuracy across all configurations, reflecting the high inter-subject variability of EEG signals.

5. **Skepticism of literature benchmarks.** Prior studies reported remarkably high accuracies on similar classification tasks, such as 91.7% (Aci et al., 2019) and an unprecedented 99.6% (Wang & Kim, 2025). Given the extreme noise inherent to raw EEG data observed in this project, these near-perfect results warrant scrutiny. Even considering their use of 32-channel, 500 Hz setups, achieving $\sim$100% accuracy on highly variable brainwave data suggests potential data leakage or exceptionally favorable train/test splits in those studies.

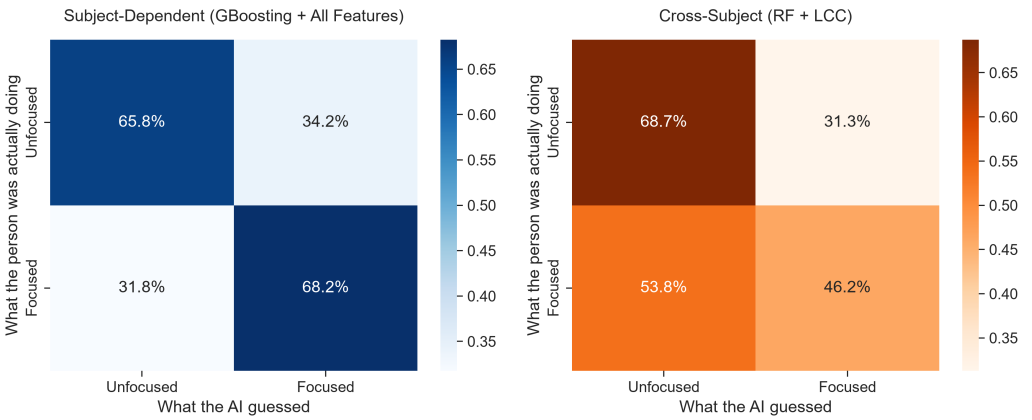

Figure 2: Aggregated confusion matrices across all 20 subjects for the top-performing configuration in each scenario: GBoosting with all 96 features (subject-dependent, 66.9%) and Random Forest with LCC-selected features (cross-subject, 58.0%).

7 Limitations and Future Work

Several limitations should be noted. First, the best cross-subject accuracy of 58.0% is only modestly above the 52.5% baseline. While the improvement is consistent across subjects, it is not yet sufficient for an out-of-the-box BCI product without prior calibration.

Most importantly, this project used research-grade data that was artificially restricted to 8 channels. A real NeuroPawn headset would introduce additional noise from dry electrodes, lower signal quality, and motion artifacts. Validating these findings on actual consumer-grade hardware is a necessary next step.

If I had more time, I would explore within this setup by trying Grid Search to improve model hyperparameters and reconsider which channels of the 32 I want to keep using feature selection. I would also try deep learning architectures such as EEGNet, which can learn spatial and temporal features directly from raw waveforms rather than relying on handcrafted PSD features.

8 Conclusion

This project built an end-to-end pipeline from raw 90 GB EEG text files to trained classifiers, simulating the constraints of a budget 8-channel headset. The results show that tree-based ensemble methods, particularly LightGBM, can distinguish focused from unfocused mental states above baseline. These findings provide NeuraXtension with a concrete starting point: LightGBM on the full PSD-derived feature set is the strongest classical baseline, but meaningful cross-subject accuracy will likely require advanced architectures like deep learning or spatial filtering.

Project Experience Summary

EEG Attention State Classifier: Brain-Computer Interface

- Built a machine learning pipeline using Python, scikit-learn, and LightGBM to process 90 GB of raw EEG data, successfully classifying mental attention states with up to 66.9% accuracy.
- Extracted 96 frequency and statistical features per epoch by applying overlapping time windows and logarithmic transformations using NumPy and SciPy, effectively doubling the usable training data to 2,000+ minutes.
- Evaluated 800 individual model configurations by systematically testing 4 classifiers and 5 feature selection methods to identify LightGBM as the strongest baseline algorithm for an 8-channel budget headset.
- Accelerated model training time by implementing joblib parallelization and LightGBM's native multi-threading across a 16-thread CPU, completing the entire cross-subject evaluation in just 1 hour.